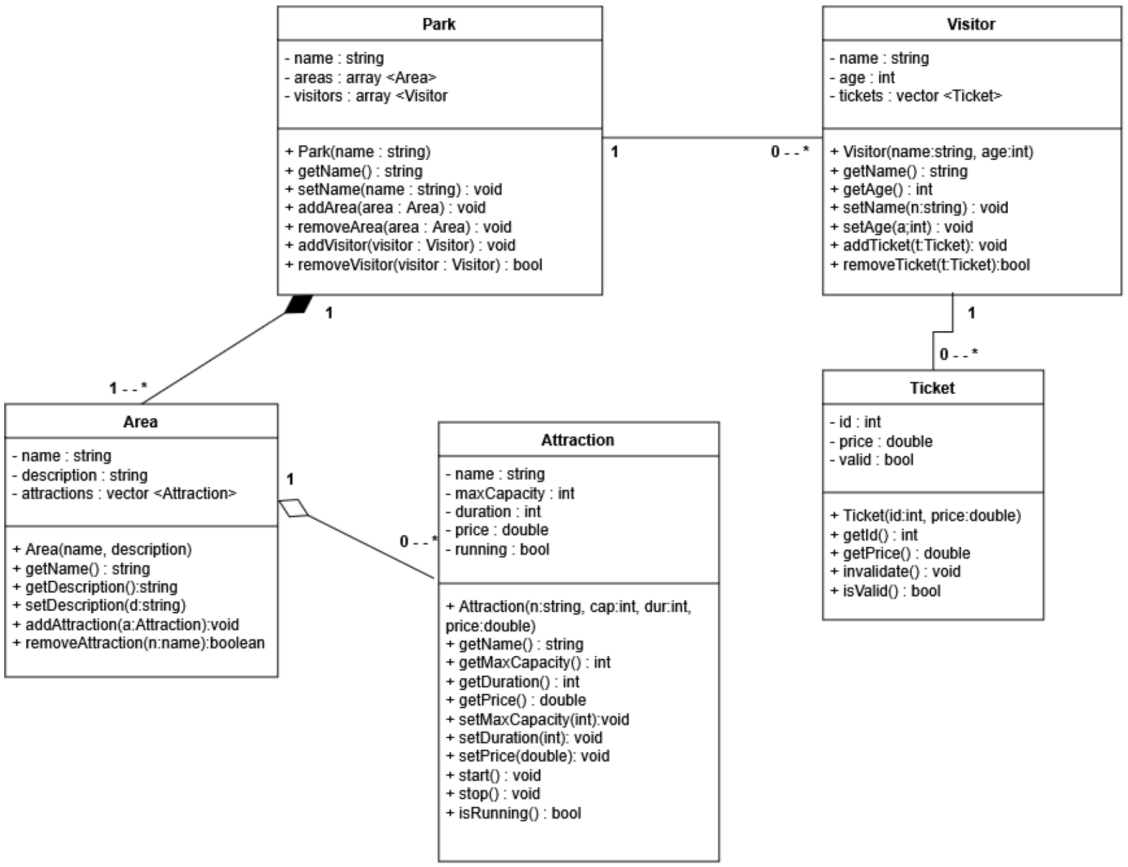
Lab 2 Report

Margalida Ferriol Torres (u266481), Martí Carrasco Guitart (u266404) - P201 / G04

1. Introducció

En aquest laboratori, l'objectiu és implementar totes les classes que s'especifiquen en el diagrama UML del Seminari 1, amb els seus atributs i mètodes corresponents. Llavors, s'ha de testejar el correcte funcionament de la implementació creant objectes amb diversos atributs, i imprimint-los per pantalla per veure que hi hagi els valors esperats. Hem utilitzat el diagrama proporcionat en el document:



2. Implementació

- **Ticket:** Té tres atributs independents: l'ID, que és un enter identificador únic per cada tiquet, un preu que no pot ser negatiu, i un booleà que representa la validesa del tiquet.

  El seu constructor inicialitza l'ID, el preu, i la validesa. Dins seu, es comprova si el preu no és negatiu amb un 'if', i si és així, s'assigna el preu especificat a l'objecte. Si el preu és negatiu, s'inicialitza amb el valor 0.0. La validesa s'inicialitza amb el valor *true*.

  També s'han implementat els *getters* de l'ID i el preu, i un *setter* per invalidar el tiquet, que força el seu atribut de validesa cap al valor *false*. No hi ha *setter* per tornar a fer vàlid un tiquet, ja que sempre comencen vàlids i només es poden invalidar una vegada.

- **Visitor**: És una classe amb dos atributs que són un *string* i un enter, i un altre atribut que emmagatzema tots els tiquets associats a aquell visitant. Així doncs, l'atribut 'tickets', és un vector que emmagatzema objectes de tipus 'Ticket'.

  Hi ha el *setter* d'age que comprova que l'edat introduïda sigui positiva, i si és negativa, no s'acaba modificant. Per al mètode que assigna un nou tiquet al visitor, s'utilitza la funció 'push_back' de la llibreria <vector>, que emmagatzema l'objecte a la posició d'índex més petit, lliure dins del vector.

  Per altra banda, hi ha el mètode "removeTicket" que elimina el tiquet que s'especifica quan es crida la funció. Per trobar la posició en la qual es troba el tiquet dins del vector per tal de poder-lo eliminar, es fa una cerca lineal pel vector, comparant els tickets pel seu ID, i quan coincideixen els ID, es fa un *erase* de la posició en què es troba el tiquet al vector. Hem utilitzat la cerca lineal, ja que el vector tickets de cada visitant sol tenir un nombre prou baix de tickets emmagatzemats, que fa que no s'hagi de demanar massa temps i recursos a l'equip. Si el projecte fos d'una escala superior, potser seria millor considerar altres algorismes per fer-ho.

- **Park**: Park és la més important perquè serveix per guardar i organitzar tot el parc en general. Té tres atributs principals: el nom del parc (name), una llista amb totes les àrees que té (areas) i una altra llista amb els visitants que hi ha registrats (visitors). Per fer-ho servir, té un constructor per crear el parc, els mètodes getName i setName per veure o canviar el seu nom, i les funcions per anar afegint o traient tant àrees (addArea, removeArea) com visitants (addVisitor, removeVisitor)

- **Area**: La classe Area serveix per agrupar diferents atraccions dins d'una mateixa zona del parc. Té tres atributs: el nom de la zona (name), una petita descripció (description) i un vector anomenat 'attractions' que guarda les atraccions que hi ha a l'àrea. Té els mètodes getName, getDescription i setDescription per veure i canviar la informació de l'àrea, i també les funcions addAttraction i removeAttraction per anar afegint o treient atraccions de la llista.

- **Attraction**: La classe Attraction serveix per representar cada una de les atraccions del parc. Té els atributs nom (name), capacitat màxima (maxCapacity), durada (duration), preu (price) i una variable booleana anomenada running per saber si està en marxa o no. El constructor s'encarrega d'inicialitzar tots aquests valors quan es crea l'atracció, comprovant que la capacitat, la durada o el preu no siguin negatius; si ho són, els canvia per valors per defecte vàlids (maxCapacity = 1, duration = 1, price = 0.0). A més, té els mètodes getters i setters per consultar o canviar la capacitat, la durada i el preu. Per controlar si funciona, utilitza les funcions start() i stop() per canviar l'atribut running, i el mètode isRunning() per comprovar quin és el seu estat en cada moment.

3. Proves realitzades

Per comprovar que totes les classes i el sistema en general funcionen com s'espera, hem fet diverses proves des del fitxer de test (main.cpp / testPark.cpp):

1. **Creació d'objectes i consulta de dades**: Hem instanciat objectes de cada classe (Ticket, Visitor, Attraction, Area i Park) amb valors vàlids i hem imprès la seva informació per pantalla per comprovar que es guardaven i es mostraven correctament.
2. **Proves amb valors no vàlids**: Hem creat atraccions, visitants i tiquets passant valors negatius (com preus, edats, capacitats o durades negatives) per verificar que els constructors assignaven els valors per defecte corresponents.
3. **Gestió de col·leccions**: Hem provat d'afegir i eliminar atraccions dins d'una Area, així com tiquets dins d'un Visitor, comprovant que la cerca lineal per ID i la funció erase eliminaven l'element correcte dins del vector.
4. **Canvis d'estat**: Hem verificat el funcionament del mètode invalidate() als tiquets per comprovar que passaven a ser no vàlids, i hem provat d'arrencar i aturar les atraccions amb start() i stop() controlant l'estat amb isRunning().

4. Conclusions

Com a conclusió, la implementació realitzada creiem que compleix amb tots els requisits i l'estructura demanada al diagrama UML del Seminari 1. El programa permet organitzar i gestionar totes les entitats del parc d'atraccions. La part que ha requerit més atenció ha estat la correcta manipulació dels vectors d'objectes i la cerca lineal. Finalment, les proves realitzades confirmen que el sistema gestiona adequadament les dades no vàlides o operacions incorrectes.